

C3b - Validation Layer

1. Schema-Driven Validation

1.1 Request Validation

Validate:

- Required/Optional fields
- Field types
- Enum values
- String formats
- Numeric constraints
- Request body structure
- Required headers

1.2 Response Validation

Validate:

- HTTP status
- Response body schema
- Required/Optional response fields
- Field types
- Enum values
- Formats
- Response headers
- Content type
- Error behaviour

1.3 Do not define undocumented business rules

Schema-driven validation should not convert undefined business rules into contractual assertions. Assumptions if any should be confirmed with Product Owner or Devs.

Example: Contract says “amount is numeric”. The contract does not say “exact precision, exact range, zero behavior, negative behavior”. So, validate what the contract defines.

Record the missing business rules as a contract gap which is already done as part of C3a.

2. Consumer-Driven Contracts

2.1 Mobile App Contract

Given account exists, the mobile contract should cover the fields it requires:

1. Request: account_id, amount, operation_type_id.
2. Response: amount, event_date, transaction_id, operation_type_id

3. Request headers: content-type
4. Response headers: request/correlation ID, content-type

It should validate:

- Field presence
- Field types
- Required headers
- Supported operations
- HTTP status
- Error response structure
- Request/Correlation ID from response headers

2.2 Ledge Service Contract

Given account exists, the ledger contract should cover the fields it requires:

1. Request: account_id, amount, operation_type_id.
2. Response: account_id, amount, event_date, transaction_id, type
3. Request headers: content-type
4. Response headers: request/correlation ID, content-type

It should validate:

- Field presence
- Field types
- Allowed transaction types
- Amount representation with sign behaviour
- HTTP status
- Backward compatibility requirements
- Request/Correlation ID from response headers

3. Preventing Provider Breaking Changes

The provider CI should verify both consumer contracts before allowing a breaking change to merge.

Provider change → Consumer compatibility verification → Mobile Contract + Ledger Contract → Merge Allowed (If no breaking changes found)

Intentional breaking changes should be made in new versions of the API rather than changing it silently.

4. Pact-Style Contract Design

4.1 Mobile App

Consumer: Mobile App

Provider: Transactions Service

Provider state: account exists

Request: POST /transactions

Body: {"account_id": 123, "amount": 100.00, "operation": "PURCHASE"}

Expected response: 201 Created

Required response fields: amount, event_date, transaction_id, operation_type_id

The provider pipeline then verifies that it satisfies the consumer-defined contract.

4.2 Ledger Service

Consumer: Ledger Service

Provider: Transactions Service

Provider state: account exists

Request: POST /transactions

Body: {"account_id": 123, "amount": 100.00, "operation": "PURCHASE"}

Expected response: 201 Created

Required response fields: account_id, amount, event_date, transaction_id, type

The provider pipeline then verifies that it satisfies the consumer-defined contract.

5. Mock Drift

5.1 Keeping The Mock Faithful

Use the versioned contract as the single source of truth for mock generation. Avoid maintaining a manual mock that duplicates the contract behaviour. This reduces the risk of: Contract v2 → Mock v1 → Tests v1. The mock should be regenerated with every contract change.

5.2 Detecting Contract Drift

Drift detection should happen at multiple levels.

5.2.1 Contract Drift

Run contract validation in CI.

Contract change → Prism Mock → Contract tests → Invalid change fails the pipeline

5.2.2 Staging Drift

Once staging is available, run the same contract suite against the real provider.

Contract change → Staging environment → Contract tests + API tests → Invalid change fails the pipeline